

코딩-규칙

코딩 규칙

이 문서는 RobomationLAB 에서 로봇 코딩을 할 때 지켜야 할 코딩 규칙에 대해 설명합니다.

참고: 스크립트 코딩은 **Python 전용**으로 동작하며, 블록은 `robomation` 파이썬 패키지를 사용하는 Python 코드로 대 변환됩니다.

순서 안내: 공통 핵심 규칙(~) → 블록 코드 규칙(~) → 파이썬 스크립트 코드 규칙(~) 순으로 정리되어 있습니다. 블록 코드와 파이썬 코드 생성 모두 중요하게 지켜야 합니다.

1. 블록 ↔ 파이썬 1대1 변환 제약

블록 코딩 에디터와 파이썬 코딩 에디터는 대 로 대응되어 실시간으로 양방향 변환됩니다.

- 블록 → 파이썬: 블록은 항상 올바른 파이썬 코드로 변환됩니다.
- 파이썬 → 블록: **블록으로 표현할 수 있는 코드**에 한해 변환됩니다.

따라서 파이썬 에디터에서 다음과 같은 코드를 작성하면, 파이썬 → 블록 코딩 에디터로 전환이 되지 않을 수 있으며, 이 경우 전환이 실패한 원인이 함께 표시됩니다.

- . 파이썬 문법 오류가 있는 코드
- . 블록으로 대응되지 않는(변환 규칙에 정의되지 않은) 함수·메서드·문법을 사용한 코드

블록 코딩 에디터로의 전환을 정상적으로 유지하려면, 이 문서와 각 로봇 문서에 정의된 클래스·메서드·함수 및 지원되는 문법 범위 내에서 코드를 작성해야 합니다.

2. 코드 제시 규칙

robomation 파이썬 API 메뉴얼과 블록 **파이썬 변환 규칙 문서**에 명시된 클래스, 메서드 및 함수만 사용할 수 있습니다.

로봇 제어 시 위 문서에 정의되지 않은 새로운 로봇 제어 메서드를 생성하여 사용하는 것은 엄격히 금지됩니다. 단, 사용자가 새로운 함수의 생성을 요청한 경우에는 가능합니다.

위 문서에 명시된 변환 예시에 나타나지 않는 임시 변수(예: 로봇 인스턴스를 담는 변수 외의 추가 변수) 선언은 사용자의 변수 생성 요청 시에만 생성할 수 있습니다.

또한, 메서드의 매개값은 반드시 리터럴로 작성해야 합니다.

```
# 올바른 예시
hamster_s.set_wheel_speed('both', 100)

# 잘못된 예시 - API 메뉴얼에 명시되지 않은 함수 사용
hamster_s.set_wheel('both', 100)

# 잘못된 예시 - 임의 변수 선언
SPEED = 100
hamster_s.set_wheel_speed('both', SPEED)
```

3. 블록 코드 기본 구조

모든 블록 코드 제시 시, 프로그램의 진입점 역할을 하는 최상위 함수 블록인 시작하기와 무한 반복하기를 기본 구조로 항상 포함하여 제시합니다.

이 규칙을 추가하면, 앞으로 모든 블록 코드는 아래와 같은 기본 구조를 갖게 됩니다.

블록 구조 (Block Composer)	제시 방법 (텍스트 형식)
시작하기	시작하기
(내부 블록)	(내부 블록)
무한 반복하기	무한 반복하기
(내부 블록)	(내부 블록)

4. 블록 코드 형식 (줄 바꿈 및 들여쓰기 규칙)

- 최상위 블록(시작하기, 무한 반복하기 등)은 왼쪽 정렬합니다.
- 각 명령어 블록은 반드시 개행 문자를 사용하여 분리하여, 한 줄에 하나의 블록만 출력해야 합니다.

- 내부 실행 영역을 가지는 블록(만약, 반복하기, 함수 정의 등)의 내부에 포함되는 하위 블록은 들여쓰기를 적용하여 계층 구조를 명확하게 표현합니다.

5. 내부 블록 및 조건 표현 규칙

드롭다운 메뉴의 선택 값 또는 입력 값은 블록의 기능적 인자(Argument)에 해당하며, 블록의 문구 내에서 해당 값이 위치하는 곳에 대괄호([])를 사용하여 직접 삽입하여 표현합니다.

이는 블록의 고유 문구와 사용자가 선택/입력한 값을 통합하여 시각적으로 재현하는 역할을 합니다.

모든 블록 코드는 블록의 고유 명칭, 드롭다운 메뉴 선택 값, 그리고 사용자가 입력한 값을 대괄호([])를 사용하여 모두 포함하는 형태로, 실제 Block Composer의 블록 모양을 텍스트로 최대한 가깝게 재현하여 제시해야 합니다.

블록 구조 (Block Composer)	제시 방법 (텍스트 형식)'''
만약 [조건] 하기 [명령] 아니라면 [명령]	만약 [조건] 하기 [명령] 아니라면 [명령]
라쿤봇: [속도] 제어 모드로 설정하기	라쿤봇: [속도] 제어 모드로 설정하기
라쿤봇: 관절 [] 속도를 [] (으)로 정하기	라쿤봇: 관절 [] 속도를 [] (으)로 정하기

6. 스크립트 코드 기본 구조

모든 스크립트 코드(Python) 제시 시, 프로그램의 진입점 역할을 하는 함수인 setup() 과 loop()를 기본 구조로 항상 포함하여 제시합니다.

또한, 로봇 제어에 필요한 클래스/유틸리티를 사용할 수 있도록 코드 최상단에 `from robomation import *` 를 포함하고, 사용할 로봇은 인스턴스로 선언합니다.

이 규칙을 추가하면, 앞으로 모든 스크립트 코드(Python)는 아래와 같은 기본 구조를 갖게 됩니다.

```
# 파이썬 코드 기본 구조
from robomation import *

# (사용할 로봇이 있다면 인스턴스로 선언, 예시)
hamster_s = HamsterS(0)

# put setup code here, to run once:
def setup():
    pass
```

```
# put control code here, to run repeatedly:
def loop():
    pass
```

7. 스크립트 코드 형식 (줄 바꿈 및 들여쓰기 규칙)

- 최상위 함수(setup, loop 등)은 왼쪽 정렬합니다.
- 줄 바꿈 시에 발생하는 들여쓰기는 반드시 **탭(\t)** 개를 기준으로 합니다. (에디터의 들여쓰기 기준과 동일)

8. 로봇 제어 규칙 (인스턴스 / 메서드)

로봇을 제어할 때에는 다음과 같은 규칙을 따라야 합니다.

- . 코드 최상단에 `from robomation import *` 를 포함합니다.
- . 사용할 로봇을 인스턴스로 선언합니다. 변수명은 로봇별로 정해진 기본 변수명을, 인덱스는 부터 부여합니다.
- . 선언한 인스턴스의 메서드를 호출하여 로봇을 제어합니다.

```
from robomation import *

hamster_s = HamsterS(0)

def setup():
    hamster_s.set_wheel_speed('both', 50)

def loop():
    pass
```

같은 종류의 로봇을 여러 대 사용하는 경우, 인덱스를 , , ... 로
늘려 선언합니다. (예: `hamster_s = HamsterS(0)` , `hamster_s_1 = HamsterS(1)`)

로봇의 바퀴 속도 설정·이동 등은 메서드 내부에서 필요한 초기화(예: 이전 이동 명령 정리)를 자동으로 처리하므로, 별도의 선행 초기화 코드를 작성할 필요가 없습니다.

9. 유틸리티 함수 호출 규칙

시간 지연, 소리 재생, 로그 출력, 색상 등 공통 유틸리티 기능은 `Utils` 클래스의 메서드를 통해 호출합니다.

```
Utils.wait(1) # (seconds)
Utils.play_sound('', 100, False)
Utils.log(0, '', '')
```

자세한 유틸리티 함수 목록은 아래 문서를 참고하세요.

- 색상
- 소리
- 제어

10. 로봇 전용 메서드 우선 사용 규칙

로봇 하드웨어(바퀴 속도, LED, 소리 등) 제어 시, 공통 유틸리티 함수보다 로봇(예: '햄스터-S') 인스턴스의 전용 메서드를 우선해서 사용해야 합니다.

예를 들어, 로봇을 이용해 소리를 내야 하는 경우, 유틸리티 함수인 `Utils.play_sound()` 보다 '햄스터-S' 인스턴스의 `sound_clip()` 메서드를 우선해서 사용해야 합니다.

```
# 햄스터 S 로봇을 통해 소리를 냅니다. (권장)
hamster_s.sound_clip('beep')

# PC / 태블릿 / 모바일 디바이스를 통해 소리를 냅니다. (비권장)
Utils.play_sound('beep', 100, False)
```

11. Import 규칙

파이썬 코드 작성 시에는, 코드 실행에 필요한 모듈만 import 합니다.

- 로봇 제어를 위해 코드 최상단에 `from robomotion import *` 를 기본적으로 포함합니다.
- `math.` 을 사용하는 코드에는 `import math` 가, `random.` 을 사용하는 코드에는 `import random` 이 자동으로 추가됩니다.
- 그 외 불필요한 모듈은 import 하지 않습니다.